

LAPORAN PRAKTIKUM STRUKTUR DATA

Algoritma Sorting



disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR.Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi hasil praktikum mata kuliah Struktur Data dan Algoritma mengenai implementasi algoritma sorting menggunakan bahasa pemrograman Java. Pada praktikum ini dilakukan penerapan algoritma Shell Sort, Quick Sort, dan Merge Sort. Melalui praktikum ini, mahasiswa dapat memahami konsep pengurutan data serta implementasinya dalam program berbasis array. Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan laporan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah pemahaman mengenai algoritma sorting.

Padang, 5 Juni 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat.....	1
BAB II	2
PEMBAHASAN	2
2.1 Pengertian Double Algoritma Sorting.....	2
2.2 Praktikum.....	2
2.2.1 Alat dan Bahan.....	2
2.2.2 Program Class InsertionSort.....	2
2.2.3 Program Class SelectioSort	5
2.2.4 Program Class BubleSort	9
2.2.5 Program Class InsertionSortGUI.....	13
BAB III.....	32
KESIMPULAN DAN SARAN	32
3.1 Kesimpulan.....	32
3.2 Saran	32
DAFTAR PUSTAKA.....	33
Penjelasan Tugas	34

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (sorting) merupakan salah satu proses penting dalam ilmu komputer yang digunakan untuk menyusun data berdasarkan urutan tertentu agar lebih mudah dikelola dan dicari. Berbagai algoritma sorting telah dikembangkan dengan karakteristik dan tingkat efisiensi yang berbeda-beda, seperti Shell Sort, Quick Sort, dan Merge Sort.

Dalam kehidupan sehari-hari, pengurutan data dapat diterapkan pada berbagai bidang, salah satunya adalah pengelolaan playlist lagu. Dengan melakukan pengurutan berdasarkan judul lagu atau durasi lagu, pengguna dapat lebih mudah menemukan dan mengelola lagu yang diinginkan.

Pada praktikum ini dilakukan implementasi algoritma sorting menggunakan bahasa pemrograman Java dengan data berupa playlist lagu yang disimpan dalam array. Program dirancang untuk menampilkan data sebelum dan sesudah proses pengurutan sehingga pengguna dapat melihat hasil kerja algoritma secara langsung.

1.2 Tujuan Praktikum

1. Memahami konsep dasar algoritma sorting.
2. Mengimplementasikan algoritma Shell Sort, Quick Sort, dan Merge Sort menggunakan bahasa Java.
3. Membuat program GUI untuk proses pengurutan data.
4. Menampilkan proses sorting langkah demi langkah.

1.3 Manfaat

Praktikum algoritma sorting ini memiliki manfaat untuk membantu mahasiswa memahami proses pengurutan data menggunakan beberapa metode sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort. Melalui praktikum ini mahasiswa dapat mengetahui cara kerja masing-masing algoritma dalam mengurutkan data secara bertahap.

BAB II

PEMBAHASAN

2.1 Pengertian Shell Sort dan Quick Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan jarak tertentu (*gap*) dalam proses pengurutan. Pada awal proses, data dibandingkan dan diurutkan berdasarkan jarak (*gap*) yang cukup besar. Nilai *gap* kemudian diperkecil secara bertahap hingga menjadi 1. Dengan cara ini, data dapat berpindah lebih cepat menuju posisi yang benar sehingga proses pengurutan menjadi lebih efisien dibandingkan Insertion Sort biasa, terutama untuk data yang berjumlah banyak.

Quick Sort adalah algoritma pengurutan yang menggunakan teknik Divide and Conquer (membagi dan menaklukkan). Algoritma ini bekerja dengan memilih satu elemen sebagai pivot, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari pivot. Setelah itu, masing-masing bagian diurutkan kembali secara rekursif hingga seluruh data terurut. Pada program ini, pemilihan pivot dilakukan menggunakan metode Median of Three, yaitu memilih nilai tengah dari elemen awal, tengah, dan akhir untuk memperoleh pivot yang lebih baik sehingga performa pengurutan menjadi lebih efisien.

2.2 Praktikum

2.2.1 Alat dan Bahan

1. Laptop/computer dengan eclipse terinstal.
2. Materi praktikum mengenai Algoritma Sorting

2.2.2 Program Class ShellSort

```
package pekan8_2511532019;

public class ShellSort_2511532019 {
    public static void shellSort_2019 (int [] A_2019) {
        int n_2019 = A_2019.length;
        int gap_2019 = n_2019/2;
        while (gap_2019>0) {
            for (int i_2019=gap_2019; i_2019<n_2019;i_2019++) {
                int temp_2019 = A_2019[i_2019];
                int j_2019=i_2019;
```

```

        while (j_2019 >= gap_2019 && A_2019[j_2019 -
gap_2019] > temp_2019) {
            A_2019[j_2019] = A_2019[j_2019 - gap_2019];
            j_2019 = j_2019 - gap_2019;
        }
        A_2019[j_2019] = temp_2019;
    }
    gap_2019 = gap_2019 / 2;
}
}

public static void main(String[] args) {
    int[] data_2019 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray(data_2019);

    shellSort_2019(data_2019);

    System.out.print("Sesudah (Shell Sort): ");
    printArray(data_2019);
}

public static void printArray(int[] arr_2019) {
    for (int i_2019 : arr_2019) System.out.print(i_2019 + " ");
    System.out.println();
}
}

```

- package pekan8_2511532019;
→ Deklarasi package tempat program disimpan.
- public class ShellSort_2511532019 {
Membuat class bernama ShellSort_2511532019.
- public static void shellSort_2019 (int [] A_2019) {
Membuat method shellSort_2019() untuk mengurutkan array menggunakan algoritma Shell Sort.
- int n_2019 = A_2019.length;
Menyimpan jumlah elemen array ke dalam variabel n_2019.
- int gap_2019 = n_2019 / 2;
Menentukan nilai gap awal, yaitu setengah dari panjang array.
- while (gap_2019 > 0) {
Perulangan dilakukan selama nilai gap masih lebih besar dari 0.
- for (int i_2019 = gap_2019; i_2019 < n_2019; i_2019++) {
Menelusuri array mulai dari indeks sebesar nilai gap.
- int temp_2019 = A_2019[i_2019];

Menyimpan nilai elemen saat ini ke variabel sementara temp_2019.

- `int j_2019=i_2019;`
Menyimpan posisi indeks saat ini ke variabel j_2019.
- `while (j_2019>=gap_2019 && A_2019[j_2019-gap_2019]>temp_2019) {`
Membandingkan elemen yang berjarak gap dengan nilai sementara.
- `A_2019[j_2019]=A_2019[j_2019-gap_2019];`
Menggeser elemen yang lebih besar ke posisi kanan.
- `j_2019=j_2019-gap_2019;`
Memindahkan indeks ke posisi sebelumnya sesuai nilai gap.
- `A_2019[j_2019]=temp_2019;`
Menempatkan nilai sementara pada posisi yang sesuai.
- `gap_2019=gap_2019/2;`
Mengurangi nilai gap menjadi setengah dari gap sebelumnya.
- `public static void main(String[] args) {`
Method utama program.
- `int[] data_2019 = {3,10,4,6,8,9,7,2,1,5};`
Membuat array data yang akan diurutkan.
- `System.out.print("Sebelum: ");`
Menampilkan tulisan "Sebelum:".
- `printArray(data_2019);`
Menampilkan isi array sebelum diurutkan.
- `shellSort_2019(data_2019);`
Memanggil method Shell Sort untuk mengurutkan data.
- `System.out.print("Sesudah (Shell Sort): ");`
Menampilkan tulisan "Sesudah (Shell Sort):".
- `printArray(data_2019);`
Menampilkan isi array setelah diurutkan.
- `public static void printArray(int[] arr_2019) {`
Method untuk menampilkan isi array.
- `for(int i_2019:arr_2019)`
Menelusuri setiap elemen array menggunakan perulangan for-each.
- `System.out.print(i_2019+" ");`

Menampilkan setiap elemen array.

- System.out.println();

Membuat baris baru setelah seluruh elemen ditampilkan.

Output

```
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.2.3 Program Class QuickSort

```
package pekan8_2511532019;

public class QuickSort_2511532019 {
    static void swap_2019 (int[] arr_2019, int i_2019, int j_2019) {
        int temp_2019 = arr_2019[i_2019];
        arr_2019[i_2019]=arr_2019[j_2019];
        arr_2019[j_2019]=temp_2019;
    }
    //metode tambahan untuk mengatur pivot menggunakan median-of-
    three
    static void medianOfThree_2019(int[] arr_2019, int low_2019, int
    high_2019) {
        int mid_2019=low_2019+(high_2019-low_2019)/2;
        //Urutkan elemen low, mid, dan high
        if (arr_2019[low_2019]>arr_2019[mid_2019]) {
            swap_2019(arr_2019,low_2019, mid_2019);
        }
        if (arr_2019[low_2019]>arr_2019[high_2019]) {
            swap_2019 (arr_2019, low_2019, high_2019);
        }
        if (arr_2019[mid_2019]>arr_2019[high_2019]) {
            swap_2019(arr_2019, mid_2019, high_2019);
        }
        swap_2019 (arr_2019,mid_2019,high_2019);
    }
    static int partition_2019(int[] arr_2019, int low_2019, int
    high_2019) {
        //panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2019(arr_2019,low_2019,high_2019);

        int pivot_2019=arr_2019[high_2019];//sekarang
        arr[high]sudah berisi nilai median
        int i_2019=(low_2019-1);

        for (int j_2019=low_2019; j_2019<=high_2019-1; j_2019++) {
            //jika elemen saat ini lebih kecil dari atau sama
            dengan pivot
            if (arr_2019[j_2019]<pivot_2019) {
                //Increment indeks elemen yang lebih kecil
                i_2019++;
                swap_2019 (arr_2019,i_2019,j_2019);
            }
        }
    }
}
```



```

    }
    }
    swap_2019 (arr_2019, i_2019+1, high_2019);
    return(i_2019+1);
}
static void quickSort_2019(int[] arr_2019, int low_2019, int
high_2019) {
    if (low_2019<high_2019) {
        int pi_2019=partition_2019(arr_2019, low_2019,
high_2019);

        quickSort_2019(arr_2019, low_2019, pi_2019-1);
        quickSort_2019(arr_2019, pi_2019+1, high_2019);
    }
}
public static void printArr_2019(int[] arr_2019) {
    for (int i_2019=0; i_2019<arr_2019.length; i_2019++) {
        System.out.print(arr_2019[i_2019]+" ");
    }
    System.out.println();
}

public static void main (String[] args) {
    int[] arr_2019= {10,7,8,9,1,5};
    int N_2019 = arr_2019.length;
    System.out.print("Data belum ditemukan: ");
    printArr_2019(arr_2019);

    quickSort_2019(arr_2019, 0, N_2019-1);

    System.out.print("Data terurut quicksort: ");
    printArr_2019(arr_2019);
}
}

```

- package pekan8_2511532019;
Deklarasi package tempat program disimpan.
- public class QuickSort_2511532019 {
Membuat class bernama QuickSort_2511532019.
- static void swap_2019 (int[] arr_2019, int i_2019, int j_2019) {
Membuat method untuk menukar posisi dua elemen array.
- int temp_2019 = arr_2019[i_2019];
Menyimpan nilai elemen pertama ke variabel sementara.
- arr_2019[i_2019]=arr_2019[j_2019];
Memindahkan nilai elemen kedua ke posisi elemen pertama.
- arr_2019[j_2019]=temp_2019;
Menempatkan nilai sementara ke posisi elemen kedua.

- `static void medianOfThree_2019(int[] arr_2019, int low_2019, int high_2019) {`
Membuat method untuk menentukan pivot menggunakan teknik *Median of Three*.
- `int mid_2019=low_2019+(high_2019-low_2019)/2;`
Menghitung indeks tengah array.
- `if (arr_2019[low_2019]>arr_2019[mid_2019]) {`
 `swap_2019(arr_2019,low_2019, mid_2019);`
 `}`
Menukar nilai jika elemen awal lebih besar dari elemen tengah.
- `if (arr_2019[low_2019]>arr_2019[high_2019]) {`
 `swap_2019 (arr_2019, low_2019, high_2019);`
 `}`
Menukar nilai jika elemen awal lebih besar dari elemen akhir.
- `if (arr_2019[mid_2019]>arr_2019[high_2019]) {`
 `swap_2019(arr_2019, mid_2019, high_2019);`
 `}`
Menukar nilai jika elemen tengah lebih besar dari elemen akhir.
- `swap_2019 (arr_2019,mid_2019,high_2019);`
Memindahkan nilai median ke posisi akhir array sebagai pivot.
- `static int partition_2019(int[] arr_2019, int low_2019, int high_2019) {`
Membuat method untuk membagi array berdasarkan pivot.
- `medianOfThree_2019(arr_2019,low_2019,high_2019);`
Memanggil method Median of Three untuk menentukan pivot.
- `int pivot_2019=arr_2019[high_2019];`
Menetapkan elemen terakhir sebagai pivot.
- `int i_2019=(low_2019-1);`
Menyiapkan indeks untuk elemen yang lebih kecil dari pivot.
- `for (int j_2019=low_2019; j_2019<=high_2019-1; j_2019++) {`
Menelusuri elemen dari awal hingga sebelum pivot.
- `if (arr_2019[j_2019]<pivot_2019) {`
Mengecek apakah elemen lebih kecil dari pivot.

- `i_2019++;`
Menambah indeks elemen kecil.
- `swap_2019 (arr_2019,i_2019,j_2019);`
Menukar elemen agar berada di sisi kiri pivot.
- `swap_2019 (arr_2019, i_2019+1, high_2019);`
Menempatkan pivot pada posisi yang benar.
- `return(i_2019+1);`
Mengembalikan posisi pivot.
- `static void quickSort_2019(int[] arr_2019, int low_2019, int high_2019) {`
Membuat method Quick Sort.
- `if (low_2019<high_2019) {`
Mengecek apakah masih terdapat lebih dari satu elemen untuk diurutkan.
- `int pi_2019=partition_2019(arr_2019, low_2019, high_2019);`
Membagi array dan mendapatkan posisi pivot.
- `quickSort_2019(arr_2019, low_2019, pi_2019-1);`
Mengurutkan bagian kiri pivot secara rekursif.
- `quickSort_2019(arr_2019, pi_2019+1, high_2019);`
Mengurutkan bagian kanan pivot secara rekursif.
- `public static void printArr_2019(int[] arr_2019) {`
Membuat method untuk menampilkan isi array.
- `for (int i_2019=0; i_2019<arr_2019.length; i_2019++) {`
Menelusuri seluruh elemen array.
- `System.out.print(arr_2019[i_2019]+" ");`
Menampilkan setiap elemen array.
- `System.out.println();`
Membuat baris baru setelah seluruh data ditampilkan.
- `public static void main (String[] args) {`
Method utama program.
- `int[] arr_2019= {10,7,8,9,1,5};`
Membuat array yang akan diurutkan.
- `int N_2019 = arr_2019.length;`
Menyimpan jumlah elemen array.

- `System.out.print("Data belum ditemukan: ");`
Menampilkan teks sebelum data diurutkan.
Sebaiknya tulisan ini diperbaiki menjadi "Data belum terurut:" karena data belum diurutkan, bukan belum ditemukan.
- `printArr_2019(arr_2019);`
Menampilkan isi array sebelum diurutkan.
- `quickSort_2019(arr_2019, 0, N_2019-1);`
Memanggil method Quick Sort untuk mengurutkan data.
- `System.out.print("Data terurut quicksort: ");`
Menampilkan teks setelah proses pengurutan selesai.
- `printArr_2019(arr_2019);`
Menampilkan isi array yang sudah terurut.

Output

```
Data belum ditemukan: 10 7 8 9 1 5
Data terurut quicksort: 1 5 7 8 9 10
```

2.2.4 Program Class MergeSort

```
package pekan8_2511532019;

public class MergeSort_2511532019 {
    void merge_2019 (int arr_2019[], int l_2019, int m_2019, int
r_2019) {
        //find sizes of two subarrays to be merged
        int n1_2019=m_2019-l_2019+1;
        int n2_2019=r_2019-m_2019;
        /* Create temp arrays*/
        int L_2019[] = new int[n1_2019];
        int R_2019[] = new int [n2_2019];
        /* Copy data to temp arrays*/
        for (int i_2019= 0; i_2019<n1_2019; ++i_2019)
            L_2019[i_2019]=arr_2019[l_2019+i_2019];
        for (int j_2019=0; j_2019<n2_2019;++j_2019)
            R_2019[j_2019]=arr_2019[m_2019+1+j_2019];
        int i_2019=0,j_2019=0;

        //initial index of merged suarray array
        int k_2019=l_2019;
        while (i_2019<n1_2019 && j_2019<n2_2019) {
            if (L_2019[i_2019]<= R_2019[j_2019]) {
                arr_2019[k_2019]=L_2019[i_2019];
                i_2019++;
            }else {
```

```

        arr_2019[k_2019]=R_2019[j_2019];
        j_2019++;
    }
    k_2019++;
}
/* Copy remaining elements of L[] if any */
while (j_2019<n2_2019) {
    arr_2019[k_2019]=R_2019[j_2019];
    j_2019++;
    k_2019++;
}
}
void sort_2019 (int arr_2019[], int l_2019, int r_2019) {
    if (l_2019<r_2019) {
        //find the middle point
        int m_2019 = (l_2019+r_2019)/2;
        //sort first and second halves
        sort_2019 (arr_2019, l_2019, m_2019);
        sort_2019 (arr_2019, m_2019+1, r_2019);
        //merge the sorted halves
        merge_2019 (arr_2019, l_2019, m_2019, r_2019);
    }
}
/*A utility function to print array of size n*/
static void printArray_2019(int arr_2019[]) {
    int n_2019 = arr_2019.length;
    for (int i_2019=0; i_2019<n_2019;++i_2019)
        System.out.print(arr_2019[i_2019]+" ");
    System.out.println();
}
public static void main (String args[]) {
    int arr_2019[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut");
    printArray_2019(arr_2019);
    MergeSort_2511532019 ob_2019=new MergeSort_2511532019();
    ob_2019.sort_2019(arr_2019, 0, arr_2019.length-1);
    System.out.println("/n Sesudah Terurut menggunakan merge
Sort");
    printArray_2019(arr_2019);
}
}

```

- package pekan8_2511532019;
Deklarasi package tempat program disimpan.
- public class MergeSort_2511532019 {
Membuat class bernama MergeSort_2511532019.
- void merge_2019 (int arr_2019[], int l_2019, int m_2019, int r_2019) {
Membuat method untuk menggabungkan dua subarray yang sudah terurut.
- int n1_2019=m_2019-l_2019+1;
Menghitung jumlah elemen subarray kiri.

- `int n2_2019=r_2019-m_2019;`
Menghitung jumlah elemen subarray kanan.
- `int L_2019[] = new int[n1_2019];`
Membuat array sementara untuk bagian kiri.
- `int R_2019[] = new int [n2_2019];`
Membuat array sementara untuk bagian kanan.
- `for (int i_2019= 0; i_2019<n1_2019; ++i_2019)`
 `L_2019[i_2019]=arr_2019[l_2019+i_2019];`
Menyalin data dari array utama ke array kiri.
- `for (int j_2019=0; j_2019<n2_2019;++j_2019)`
 `R_2019[j_2019]=arr_2019[m_2019+1+j_2019];`
Menyalin data dari array utama ke array kanan.
- `int i_2019=0,j_2019=0;`
Menyiapkan indeks awal untuk array kiri dan kanan.
- `int k_2019=l_2019;`
Menyiapkan indeks untuk array hasil penggabungan.
- `while (i_2019<n1_2019 && j_2019<n2_2019) {`
Membandingkan elemen pada array kiri dan kanan.
- `if (L_2019[i_2019]<= R_2019[j_2019]) {`
Mengecek apakah elemen kiri lebih kecil atau sama dengan elemen kanan.
- `arr_2019[k_2019]=L_2019[i_2019];`
Memasukkan elemen kiri ke array utama.
- `i_2019++;`
Memindahkan indeks array kiri.
- `}else {`
 `arr_2019[k_2019]=R_2019[j_2019];`
Memasukkan elemen kanan ke array utama.
- `j_2019++;`
Memindahkan indeks array kanan.
- `}`
 `k_2019++;`
Memindahkan indeks array hasil.

- while (j_2019<n2_2019) {
Mengecek apakah masih ada sisa elemen pada array kanan.
- arr_2019[k_2019]=R_2019[j_2019];
Memasukkan sisa elemen array kanan ke array utama.
- j_2019++;
k_2019++;
Memindahkan indeks array kanan dan array hasil.
- while(i_2019<n1_2019)
untuk menyalin sisa elemen array kiri apabila masih ada data yang belum dipindahkan.
- void sort_2019 (int arr_2019[], int l_2019, int r_2019) {
Membuat method utama Merge Sort.
- if (l_2019<r_2019) {
Mengecek apakah array masih dapat dibagi.
- int m_2019 = (l_2019+r_2019)/2;
Menentukan posisi tengah array.
- sort_2019 (arr_2019, l_2019, m_2019);
Mengurutkan bagian kiri secara rekursif.
- sort_2019 (arr_2019, m_2019+1, r_2019);
Mengurutkan bagian kanan secara rekursif.
- merge_2019 (arr_2019, l_2019, m_2019, r_2019);
Menggabungkan kedua bagian yang sudah terurut.
- static void printArray_2019(int arr_2019[]) {
Membuat method untuk menampilkan isi array.
- int n_2019 = arr_2019.length;
Menyimpan jumlah elemen array.
- for (int i_2019=0; i_2019<n_2019;++i_2019)
Menelusuri seluruh elemen array.
- System.out.print(arr_2019[i_2019]+" ");
Menampilkan setiap elemen array.
- System.out.println();
Membuat baris baru setelah seluruh elemen ditampilkan.

- `public static void main (String args[]) {`
Method utama program.
- `int arr_2019[] = {12, 11, 13, 5, 6, 7};`
Membuat array yang akan diurutkan.
- `System.out.println("Sebelum terurut");`
Menampilkan teks sebelum proses sorting.
- `printArray_2019(arr_2019);`
Menampilkan isi array sebelum diurutkan.
- `MergeSort_2511532019 ob_2019=new MergeSort_2511532019();`
Membuat objek dari class MergeSort_2511532019.
- `ob_2019.sort_2019(arr_2019, 0, arr_2019.length-1);`
Memanggil method Merge Sort untuk mengurutkan array.
- `System.out.println("\nSesudah Terurut menggunakan Merge Sort");`
Menampilkan teks setelah proses sorting.
- `printArray_2019(arr_2019);`
Menampilkan array yang sudah terurut.

Output

```
Sebelum terurut
12 11 13 5 6 7
/n Sesudah Terurut menggunakan merge Sort
5 6 7 5 6 7
```

2.2.5 Program Class BubbleSortGUI

```
package pekan8_2511532019;

import java.awt.BorderLayout;

public class BubbleSortGUI_2511532019 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_2019;
    private JLabel[] labelArray_2019;
    private JButton stepButton_2019, resetButton_2019,
setButton_2019;
    private JTextField inputField_2019;
    private JPanel panelArray_2019;
    private JTextArea stepArea_2019;
    private int i_2019=1, j_2019;
    private boolean sorting_2019 = false;
```



```

private int stepCount_2019=1;

/**
 * Create the frame.
 */
public BubbleSortGUI_2511532019() {
    setTitle("Bubble Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    //panel input
    JPanel inputPanel_2019 = new JPanel (new FlowLayout());
    inputField_2019=new JTextField(30);
    setButton_2019=new JButton ("Set Array");
    inputPanel_2019.add(new JLabel ("Masukkan angka (pisahkan
dengan koma): "));
    inputPanel_2019.add (inputField_2019);
    inputPanel_2019.add(setButton_2019);

    //panel array visual
    panelArray_2019=new JPanel();
    panelArray_2019.setLayout(new FlowLayout());

    //panel kontrol
    JPanel controlPanel_2019=new JPanel();
    stepButton_2019=new JButton ("Langkah selanjutnya");
    resetButton_2019=new JButton ("Reset");
    stepButton_2019.setEnabled(false);
    controlPanel_2019.add(stepButton_2019);
    controlPanel_2019.add(resetButton_2019);

    //Area teks untuk log langkah-langkah
    stepArea_2019 = new JTextArea(8,60);
    stepArea_2019.setEditable(false);
    stepArea_2019.setFont(new Font("Monospaced", Font.PLAIN,
14));

    JScrollPane scrollPane_2019=new JScrollPane(stepArea_2019);

    //tambahkan panel ke frame
    add(inputPanel_2019, BorderLayout.NORTH);
    add(panelArray_2019, BorderLayout.CENTER);
    add(controlPanel_2019, BorderLayout.SOUTH);
    add(scrollPane_2019,BorderLayout.EAST);

    //Event Set Array
    setButton_2019.addActionListener(e-> setArrayFromInput());

    //event langkah selanjutnya
    stepButton_2019.addActionListener(e->performStep());

    //event Reset
    resetButton_2019.addActionListener(e->reset());
}
private void setArrayFromInput() {
    String text=inputField_2019.getText().trim();

```

```

        if (text.isEmpty()) return;
        String[] parts=text.split(",");
        array_2019= new int [parts.length];
        try {
            for (int k_2019=0; k_2019<parts.length;k_2019++) {
                array_2019
[k_2019]=Integer.parseInt(parts[k_2019].trim());
            }
        }catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan"+ "dengan koma!", "error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2019=0;
        j_2019=0;
        stepCount_2019=1;
        sorting_2019=true;
        stepButton_2019.setEnabled(true);
        stepArea_2019.setText("");
        panelArray_2019.removeAll();
        labelArray_2019=new JLabel[array_2019.length];
        for (int k_2019=0; k_2019<array_2019.length;k_2019++) {
            labelArray_2019[k_2019]=new JLabel
(String.valueOf(array_2019[k_2019]));
            labelArray_2019[k_2019].setFont(new Font ("Arial",
Font.BOLD,24));

            labelArray_2019[k_2019].setOpaque(true);
            labelArray_2019[k_2019].setBackground(Color.WHITE);

            labelArray_2019[k_2019].setBorder(BorderFactory.createLineBorder(
Color.BLACK));
            labelArray_2019[k_2019].setPreferredSize (new
Dimension(50,50));

            labelArray_2019[k_2019].setHorizontalAlignment(SwingConstants.CEN
TER);

            panelArray_2019.add(labelArray_2019[k_2019]);
        }
        panelArray_2019.revalidate();
        panelArray_2019.repaint();
    }

    private void performStep() {
        if (!sorting_2019||i_2019>=array_2019.length-1) {
            sorting_2019 = false;
            stepButton_2019.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
            return;}
        resetHighlights_2019();
        StringBuilder stepLog_2019=new StringBuilder();
        labelArray_2019[j_2019].setBackground(Color.CYAN);
        labelArray_2019[j_2019+1].setBackground(Color.CYAN);
        if(array_2019[j_2019]>array_2019[j_2019+1]) {
            //swap
            int temp_2019=array_2019[j_2019];

```

```

        array_2019[j_2019]=array_2019[j_2019+1];
        array_2019[j_2019+1]=temp_2019;
        labelArray_2019[j_2019].setBackground(Color.RED);
        labelArray_2019[j_2019+1].setBackground(Color.RED);
        stepLog_2019.append("Langkah ").append
(stepCount_2019).append(": Menukar elemen ke-")

        .append(j_2019).append("(").append(array_2019[j_2019+1]).append("
) dengan ke-")

        .append(j_2019+1).append("(").append
(array_2019[j_2019]).append(")\n");
    }else {
        stepLog_2019.append("Langkah").append
(stepCount_2019).append(": Tidak ada pertukaran antara ke-")
        .append(j_2019).append("dan ke-
").append(j_2019+1).append("\n");
        stepLog_2019.append ("Hasil:
").append(arrayToString(array_2019)).append("\n\n");
        stepArea_2019.append(stepLog_2019.toString());
        updateLabels();
        j_2019++;
        if(j_2019>=array_2019.length-i_2019-1) {
            j_2019=0;
            i_2019++;}
        stepCount_2019++;
        if(i_2019>=array_2019.length-1) {
            sorting_2019=false;
            stepButton_2019.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai !");
        }
    }
}

private void updateLabels() {
    for (int k_2019=0; k_2019< array_2019.length; k_2019++) {

        labelArray_2019[k_2019].setText(String.valueOf(array_2019[k_2019]
));
    }
}

private void resetHighlights_2019() {
    for(JLabel label : labelArray_2019) {
        label.setBackground(Color.WHITE);
    }
}

private void reset () {
    inputField_2019.setText("");
    panelArray_2019.removeAll();
    panelArray_2019.revalidate();
    panelArray_2019.repaint();
    stepArea_2019.setText("");
    stepButton_2019.setEnabled(false);
    sorting_2019=false;
    i_2019=0;
    j_2019=0;
    stepCount_2019=1;

```

```

    }

    private String arrayToString(int[] arr) {
        StringBuilder sb_2019= new StringBuilder();
        for (int k_2019=0; k_2019<arr.length;k_2019++) {
            sb_2019.append (arr[k_2019]);
            if (k_2019<arr.length-1) sb_2019.append(", ");
        }
        return sb_2019.toString();
    }

    public static void main (String[] args) {
        SwingUtilities.invokeLater(()->{
            BubbleSortGUI_2511532019 gui=new
BubbleSortGUI_2511532019();
            gui.setVisible(true);
        });
    }
}

```

- import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Dimension;
import java.awt.FlowLayout;
import javax.swing.*;
Mengimpor library AWT dan Swing yang diperlukan untuk membuat tampilan GUI seperti frame, panel, tombol, label, area teks, warna, ukuran komponen, dan pengaturan tata letak.
- public class BubbleSortGUI_2511532019 extends JFrame {
Membuat class GUI Bubble Sort yang mewarisi JFrame sehingga dapat digunakan sebagai jendela aplikasi.
- private int[] array_2019;
private JLabel[] labelArray_2019;
private JButton stepButton_2019, resetButton_2019, setButton_2019;
private JTextField inputField_2019;
private JPanel panelArray_2019;
private JTextArea stepArea_2019;
private int i_2019=1, j_2019;
private boolean sorting_2019=false;
private int stepCount_2019=1;

Mendeklarasikan variabel yang digunakan untuk menyimpan data array, komponen GUI, indeks proses Bubble Sort, status sorting, dan pencacah langkah.

- `public BubbleSortGUI_2511532019() {`
Membuat dan mengatur seluruh tampilan GUI seperti ukuran jendela, panel input, panel visualisasi array, tombol kontrol, serta area log langkah-langkah sorting.
`}`
- `JPanel inputPanel_2019 = new JPanel(new FlowLayout());`
`inputField_2019 = new JTextField(30);`
`setButton_2019 = new JButton("Set Array");`
Membuat panel untuk memasukkan data array dan tombol untuk memproses input.
- `panelArray_2019 = new JPanel();`
`panelArray_2019.setLayout(new FlowLayout());`
Membuat panel yang digunakan untuk menampilkan elemen-elemen array secara visual.
- `stepButton_2019 = new JButton("Langkah selanjutnya");`
`resetButton_2019 = new JButton("Reset");`
Membuat tombol untuk menjalankan proses Bubble Sort langkah demi langkah dan mengembalikan program ke kondisi awal.
- `stepArea_2019 = new JTextArea(8,60);`
`stepArea_2019.setEditable(false);`
`stepArea_2019.setFont(new Font("Monospaced", Font.PLAIN, 14));`
Membuat area teks untuk menampilkan informasi setiap langkah proses pengurutan.
- `setButton_2019.addActionListener(e-> setArrayFromInput());`
`stepButton_2019.addActionListener(e->performStep());`
`resetButton_2019.addActionListener(e->reset());`
Menghubungkan tombol dengan fungsi yang akan dijalankan ketika tombol ditekan.
- `private void setArrayFromInput()`
Method untuk membaca input array dari pengguna.

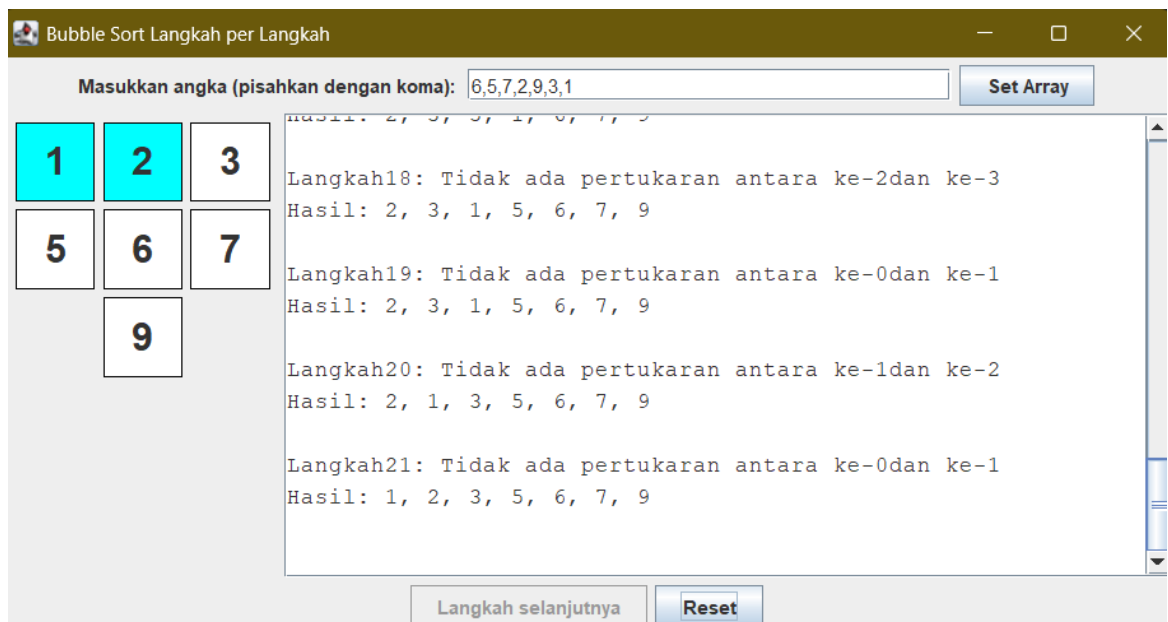
- `String text=inputField_2019.getText().trim();`
Mengambil teks dari JTextField dan menghapus spasi di awal maupun akhir.
- `String[] parts=text.split(",");`
Memisahkan data berdasarkan tanda koma.
- `array_2019=new int[parts.length];`
Membuat array sesuai jumlah data yang dimasukkan.
- `array_2019[k_2019]=Integer.parseInt(parts[k_2019].trim());`
Mengubah data String menjadi integer.
- `stepButton_2019.setEnabled(true);`
Mengaktifkan tombol langkah selanjutnya.
- `labelArray_2019[k_2019]=new
JLabel(String.valueOf(array_2019[k_2019]));`
Membuat label untuk menampilkan setiap elemen array.
- `panelArray_2019.add(labelArray_2019[k_2019]);`
Menambahkan label ke panel visualisasi.
- `private void performStep()`
Method untuk menjalankan satu langkah Bubble Sort.
- `resetHighlights_2019();`
Menghapus warna penanda pada langkah sebelumnya.
- `labelArray_2019[j_2019].setBackground(Color.CYAN);`
`labelArray_2019[j_2019+1].setBackground(Color.CYAN);`
Memberi warna pada dua elemen yang sedang dibandingkan.
- `if(array_2019[j_2019]>array_2019[j_2019+1])`
Mengecek apakah elemen kiri lebih besar daripada elemen kanan.
- `int temp_2019=array_2019[j_2019];`
Menyimpan sementara nilai array saat proses pertukaran.
- `array_2019[j_2019]=array_2019[j_2019+1];`
`array_2019[j_2019+1]=temp_2019;`
Menukar posisi dua elemen.
- `stepLog_2019.append(...)`
Menambahkan informasi langkah ke area log.

- `updateLabels();`
Memperbarui tampilan array setelah pertukaran.
- `j_2019++;`
Berpindah ke pasangan elemen berikutnya.
- `i_2019++;`
Berpindah ke iterasi Bubble Sort berikutnya.
- `private void updateLabels()`
Memperbarui tampilan array pada GUI setelah terjadi perubahan data.
- `labelArray_2019[k_2019].setText(String.valueOf(array_2019[k_2019]));`
Mengubah nilai yang ditampilkan pada label sesuai isi array terbaru.
- `private void resetHighlights_2019()`
Mengembalikan warna seluruh elemen array ke kondisi normal sebelum langkah berikutnya dijalankan.
- `label.setBackground(Color.WHITE);`
Mengubah warna latar label menjadi putih.
- `private void reset()`
Menghapus seluruh data input, area visualisasi, area log, serta mengembalikan variabel program ke kondisi awal.
- `inputField_2019.setText("");`
Mengosongkan input pengguna.
- `panelArray_2019.removeAll();`
Menghapus seluruh visualisasi array.
- `stepArea_2019.setText("");`
Menghapus log langkah-langkah.
- `sorting_2019=false;`
Menghentikan proses sorting.
- `stepCount_2019=1;`
Mengembalikan nomor langkah ke awal.
- `private String arrayToString(int[] arr)`
Mengubah isi array menjadi bentuk teks agar dapat ditampilkan pada area log langkah-langkah sorting.
- `StringBuilder sb_2019=new StringBuilder();`

Membuat objek untuk menyusun teks.

- `sb_2019.append(arr[k_2019]);`
Menambahkan elemen array ke dalam String.
- `return sb_2019.toString();`
Mengembalikan hasil dalam bentuk String.
- `public static void main(String[] args)`
Menjalankan program GUI Bubble Sort dan menampilkan jendela aplikasi kepada pengguna.
- `SwingUtilities.invokeLater(...)`
Menjalankan GUI pada Event Dispatch Thread Swing.
- `BubbleSortGUI_2511532019 gui=`
`new BubbleSortGUI_2511532019();`
Membuat objek GUI.
- `gui.setVisible(true);`
Menampilkan jendela program ke layar.

Output



2.2.6 Program Class MergeSortGUI

```
package pekan8_2511532019;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Dimension;
import java.awt.FlowLayout;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JScrollPane;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JTextArea;
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.util.Queue;
import java.util.LinkedList;

public class MergeSortGUI_2511532019 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int [] array_2019;
    private JLabel[] labelArray_2019;
    private JButton stepButton_2019, resetButton_2019,
setButton_2019;
    private JTextField inputField_2019;
    private JPanel panelArray_2019;
    private JTextArea stepArea_2019;
    private int i_2019=1, j_2019;
    private int stepCount_2019=1;
    private Queue<int[]> mergeQueue_2019 = new LinkedList<>();
    private boolean isMerging_2019;
    private boolean copying_2019;
    private int left_2019;
    private int mid_2019;
    private int right_2019;
    private int[] temp_2019;
    private int k_2019;

    /**
     * Create the frame.
     */
    public MergeSortGUI_2511532019() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //panel input
        JPanel inputPanel_2019 = new JPanel (new FlowLayout());
        inputField_2019=new JTextField(30);
```

```

        setButton_2019=new JButton ("Set Array");
        inputPanel_2019.add(new JLabel ("Masukkan angka (pisahkan
dengan koma): "));
        inputPanel_2019.add (inputField_2019);
        inputPanel_2019.add(setButton_2019);

        //panel array visual
        panelArray_2019=new JPanel();
        panelArray_2019.setLayout(new FlowLayout());

        //panel kontrol
        JPanel controlPanel_2019=new JPanel();
        stepButton_2019=new JButton ("Langkah selanjutnya");
        resetButton_2019=new JButton ("Reset");
        stepButton_2019.setEnabled(false);
        controlPanel_2019.add(stepButton_2019);
        controlPanel_2019.add(resetButton_2019);

        //Area teks untuk log langkah-langkah
        stepArea_2019 = new JTextArea(8,60);
        stepArea_2019.setEditable(false);
        stepArea_2019.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scrollPane_2019=new JScrollPane(stepArea_2019);

        //tambahkan panel ke frame
        add(inputPanel_2019, BorderLayout.NORTH);
        add(panelArray_2019, BorderLayout.CENTER);
        add(controlPanel_2019, BorderLayout.SOUTH);
        add(scrollPane_2019,BorderLayout.EAST);

        //Event Set Array
        setButton_2019.addActionListener(e->
setArrayFromInput_2019());

        //event langkah selanjutnya
        stepButton_2019.addActionListener(e->performStep_2019());

        //event Reset
        resetButton_2019.addActionListener(e->reset_2019());
    }
    private void setArrayFromInput_2019() {
        String text=inputField_2019.getText().trim();
        if (text.isEmpty()) return;
        String[] parts=text.split(",");
        array_2019= new int [parts.length];
        try {
            for (int k_2019=0; k_2019<parts.length;k_2019++) {
                array_2019
[k_2019]=Integer.parseInt(parts[k_2019].trim());
            }
        }catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan"+ "dengan koma!", "error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

```

```

        labelArray_2019=new JLabel[array_2019.length];
        panelArray_2019.removeAll();
        for (int i_2019=0; i_2019<array_2019.length;i_2019++) {
            labelArray_2019[i_2019]=new JLabel
(String.valueOf(array_2019[i_2019]));
            labelArray_2019[i_2019].setFont(new Font ("Arial",
Font.BOLD,24));

            labelArray_2019[i_2019].setOpaque(true);
            labelArray_2019[i_2019].setBackground(Color.WHITE);

            labelArray_2019[i_2019].setBorder(BorderFactory.createLineBorder(
Color.BLACK));
            labelArray_2019[i_2019].setPreferredSize (new
Dimension(50,50));

            labelArray_2019[i_2019].setHorizontalAlignment(SwingConstants.CEN
TER);

            panelArray_2019.add(labelArray_2019[i_2019]);
        }
        mergeQueue_2019.clear();
        generateMergeSteps(0,array_2019.length-1);
        stepButton_2019.setEnabled(true);
        stepArea_2019.setText("");
        stepCount_2019=1;
        isMerging_2019=false;
        panelArray_2019.revalidate();
        panelArray_2019.repaint();
    }
    private void generateMergeSteps(int left_2019, int right_2019) {

        if (left_2019 >= right_2019) {
            return;
        }

        int mid_2019 = (left_2019 + right_2019) / 2;

        generateMergeSteps(left_2019, mid_2019);
        generateMergeSteps(mid_2019 + 1, right_2019);

        mergeQueue_2019.add(
            new int[] {left_2019, mid_2019, right_2019}
        );
    }

    private void performStep_2019() {
        resetHighlights_2019();

        if (!isMerging_2019 && !mergeQueue_2019.isEmpty()) {
            int[] range_2019 = mergeQueue_2019.poll();
            left_2019=range_2019[0];
            mid_2019=range_2019[1];
            right_2019=range_2019[2];
            temp_2019=new int [right_2019-left_2019+1];
            i_2019=left_2019;
            j_2019=mid_2019+1;
            k_2019=0;

```

```

        copying_2019=false;
        isMerging_2019=true;
        stepArea_2019.append("Langkah "+stepCount_2019++ + ":
Mulai merge dari "+left_2019+" ke "+right_2019+ "\n");
        return;
    }
    if (isMerging_2019&&!copying_2019) {
        if (i_2019<= mid_2019 && j_2019<=right_2019) {

            labelArray_2019[i_2019].setBackground(Color.CYAN);

            labelArray_2019[j_2019].setBackground(Color.CYAN);
            if (array_2019[i_2019]<= array_2019[j_2019])
{
                temp_2019[k_2019++]=array_2019[i_2019++];
            }else {

                temp_2019[k_2019++]=array_2019[j_2019++];
            }
            stepArea_2019.append("Langkah
"+stepCount_2019++ + ": Bandingkan dan salin elemen \n");
            return;
        }else if (i_2019<=mid_2019) {
            temp_2019[k_2019++]=array_2019[i_2019++];
            stepArea_2019.append("Langkah
"+stepCount_2019++ + ": Salin sisa kiri\n");
            return;
        }else if(j_2019<=right_2019) {
            temp_2019 [k_2019++]=array_2019[j_2019++];
            stepArea_2019.append("Langkah
"+stepCount_2019++ + ": Salin sisa kanan \n");
            return;
        }else {
            copying_2019=true;
            k_2019=0;
            return;
        }
    }
    if(copying_2019 && k_2019<temp_2019.length) {
        array_2019[left_2019+k_2019]=temp_2019[k_2019];

        labelArray_2019[left_2019+k_2019].setText(String.valueOf(temp_201
9[k_2019]));

        labelArray_2019[left_2019+k_2019].setBackground(Color.GREEN);
        k_2019++;
        stepArea_2019.append("Langkah "+ stepCount_2019++ +
": Tempelkan ke array utama \n");
        return;
    }
    if (copying_2019 && k_2019==temp_2019.length) {
        isMerging_2019=false;
        copying_2019=false;
    }
    if (mergeQueue_2019.isEmpty()&&!isMerging_2019) {
        stepArea_2019.append("Selesai.\n");
    }

```

```

        stepButton_2019.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort
selesail!");
    }
}
private void resetHighlights_2019() {
    if (labelArray_2019==null)return;
    for (JLabel label : labelArray_2019) {
        label.setBackground(Color.WHITE);
    }
}
private void reset_2019() {
    inputField_2019.setText("");
    panelArray_2019.removeAll();
    panelArray_2019.revalidate();
    panelArray_2019.repaint();
    stepArea_2019.setText("");
    stepButton_2019.setEnabled(false);
    mergeQueue_2019.clear();
    isMerging_2019=false;
    stepCount_2019=1;
}
public static void main (String[] args) {
    SwingUtilities.invokeLater(()->{
        MergeSortGUI_2511532019 gui=new
MergeSortGUI_2511532019();
        gui.setVisible(true);
    });
}
}

```

- import java.awt.BorderLayout;
- import java.awt.Color;
- import java.awt.Font;
- import java.awt.Dimension;
- import java.awt.FlowLayout;
- import javax.swing.JLabel;
- import javax.swing.JOptionPane;
- import javax.swing.BorderFactory;
- import javax.swing.JButton;
- import javax.swing.JScrollPane;
- import javax.swing.JTextField;
- import javax.swing.SwingConstants;
- import javax.swing.SwingUtilities;
- import javax.swing.JTextArea;

```
import javax.swing.JFrame;
```

```
import javax.swing.JPanel;
```

```
import java.util.Queue;
```

```
import java.util.LinkedList;
```

Mengimpor library AWT, Swing, serta Queue dan LinkedList yang diperlukan untuk membuat tampilan GUI dan proses Merge Sort langkah demi langkah.

- ```
public class MergeSortGUI_2511532019 extends JFrame
```

  
Membuat class GUI Merge Sort yang mewarisi JFrame sehingga dapat digunakan sebagai jendela aplikasi.
- ```
private int [] array_2019;
```



```
private JLabel[] labelArray_2019;
```



```
private JButton stepButton_2019, resetButton_2019, setButton_2019;
```



```
private JTextField inputField_2019;
```



```
private JPanel panelArray_2019;
```



```
private JTextArea stepArea_2019;
```


Mendeklarasikan variabel untuk menyimpan data array dan komponen GUI.
- ```
private Queue<int[]> mergeQueue_2019 = new LinkedList<>();
```

  
Membuat antrian yang digunakan untuk menyimpan urutan proses merge.
- ```
private boolean isMerging_2019;
```



```
private boolean copying_2019;
```


Menyimpan status proses penggabungan data.
- ```
private int left_2019;
```

```
private int mid_2019;
```

```
private int right_2019;
```

```
private int[] temp_2019;
```

```
private int k_2019;
```

  
Menyimpan indeks dan array sementara yang digunakan saat proses merge.
- ```
public MergeSortGUI_2511532019()
```


Membuat dan mengatur seluruh tampilan GUI Merge Sort.

- `JPanel inputPanel_2019 = new JPanel(new FlowLayout());`
`inputField_2019 = new JTextField(30);`
`setButton_2019 = new JButton("Set Array");`
Membuat panel input, kotak teks, dan tombol untuk memasukkan data array.
- `panelArray_2019 = new JPanel();`
`panelArray_2019.setLayout(new FlowLayout());`
Membuat panel untuk menampilkan elemen array secara visual.
- `stepButton_2019 = new JButton("Langkah selanjutnya");`
`resetButton_2019 = new JButton("Reset");`
Membuat tombol langkah berikutnya dan tombol reset.
- `stepArea_2019 = new JTextArea(8,60);`
`stepArea_2019.setEditable(false);`
`stepArea_2019.setFont(new Font("Monospaced", Font.PLAIN, 14));`
Membuat area teks untuk menampilkan log langkah-langkah Merge Sort.
- `setButton_2019.addActionListener(e-> setArrayFromInput_2019());`
`stepButton_2019.addActionListener(e->performStep_2019());`
`resetButton_2019.addActionListener(e->reset_2019());`
Menghubungkan tombol dengan method yang akan dijalankan saat tombol ditekan.
- `private void setArrayFromInput_2019()`
Method untuk membaca data array dari pengguna.
- `String text=inputField_2019.getText().trim();`
Mengambil data dari kotak input dan menghapus spasi yang tidak diperlukan.
- `String[] parts=text.split(",");`
Memisahkan data berdasarkan tanda koma.
- `array_2019= new int [parts.length];`
Membuat array sesuai jumlah data yang dimasukkan.
- `array_2019[k_2019]=Integer.parseInt(parts[k_2019].trim());`
Mengubah data String menjadi integer.
- `labelArray_2019=new JLabel[array_2019.length];`

Membuat array label untuk visualisasi data.

- `panelArray_2019.add(labelArray_2019[i_2019]);`
Menambahkan label ke panel visualisasi.
- `mergeQueue_2019.clear();`
Menghapus seluruh data antrian merge sebelumnya.
- `generateMergeSteps(0,array_2019.length-1);`
Membuat urutan proses merge yang akan dijalankan.
- `private void generateMergeSteps(int left_2019, int right_2019)`
Method rekursif untuk membagi array menjadi beberapa bagian sebelum proses merge.
- `int mid_2019 = (left_2019 + right_2019) / 2;`
Menentukan titik tengah array.
- `generateMergeSteps(left_2019, mid_2019);`
`generateMergeSteps(mid_2019 + 1, right_2019);`
Membagi bagian kiri dan kanan array secara rekursif.
- `mergeQueue_2019.add(new int[] {left_2019, mid_2019, right_2019});`
Menyimpan informasi proses merge ke dalam antrian.
- `private void performStep_2019()`
Method untuk menjalankan satu langkah Merge Sort.
- `resetHighlights_2019();`
Menghapus warna penanda dari langkah sebelumnya.
- `int[] range_2019 = mergeQueue_2019.poll();`
Mengambil data merge berikutnya dari antrian.
- `temp_2019=new int[right_2019-left_2019+1];`
Membuat array sementara untuk menyimpan hasil penggabungan.
- `if (array_2019[i_2019] <= array_2019[j_2019])`
Membandingkan elemen dari bagian kiri dan kanan.
- `temp_2019[k_2019++]=array_2019[i_2019++];`
Menyalin elemen kiri ke array sementara.
- `temp_2019[k_2019++]=array_2019[j_2019++];`
Menyalin elemen kanan ke array sementara.
- `array_2019[left_2019+k_2019]=temp_2019[k_2019];`

Menyalin hasil merge ke array utama.

- `labelArray_2019[left_2019+k_2019].setText(String.valueOf(temp_2019[k_2019]));`

Memperbarui tampilan data pada label.

- `stepArea_2019.append(...)`

Menambahkan informasi langkah ke area log.

- `private void resetHighlights_2019()`

Method untuk mengembalikan warna seluruh elemen ke kondisi normal.

- `label.setBackground(Color.WHITE);`

Mengubah warna latar label menjadi putih.

- `private void reset_2019()`

Method untuk mengembalikan program ke kondisi awal.

- `inputField_2019.setText("");`

Mengosongkan kotak input.

- `panelArray_2019.removeAll();`

Menghapus seluruh visualisasi array.

- `stepArea_2019.setText("");`

Menghapus seluruh log langkah-langkah.

- `mergeQueue_2019.clear();`

Menghapus seluruh antrian proses merge.

- `public static void main(String[] args)`

Method utama untuk menjalankan program.

- `SwingUtilities.invokeLater(...)`

Menjalankan GUI pada Event Dispatch Thread Swing.

- `MergeSortGUI_2511532019 gui = new MergeSortGUI_2511532019();`

Membuat objek GUI Merge Sort.

- `gui.setVisible(true);`

Menampilkan jendela program ke layar.

Output

Merge Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

1	2	3
4	6	7
	9	

Langkah 34: Bandingkan dan salin elemen
Langkah 35: Bandingkan dan salin elemen
Langkah 36: Bandingkan dan salin elemen
Langkah 37: Bandingkan dan salin elemen
Langkah 38: Bandingkan dan salin elemen
Langkah 39: Salin sisa kanan
Langkah 40: Tempelkan ke array utama
Langkah 41: Tempelkan ke array utama
Langkah 42: Tempelkan ke array utama
Langkah 43: Tempelkan ke array utama
Langkah 44: Tempelkan ke array utama
Langkah 45: Tempelkan ke array utama
Langkah 46: Tempelkan ke array utama
Selesai.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih teratur dan mudah diproses. Pada praktikum ini digunakan tiga algoritma sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort. Program berhasil dibuat menggunakan Java, sehingga pengguna dapat memasukkan data, memilih metode sorting, dan melihat proses pengurutan langkah demi langkah.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan Algoritma Sorting agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class Sorting

```
package pekan8_2511532019;

import java.util.Scanner;

class Lagu_2019 {

    String judul_2019;
    String penyanyi_2019;
    int durasi_2019;

    public Lagu_2019(
        String judul_2019,
        String penyanyi_2019,
        int durasi_2019) {

        this.judul_2019 = judul_2019;
        this.penyanyi_2019 = penyanyi_2019;
        this.durasi_2019 = durasi_2019;
    }
}

public class Sorting_2511532019 {

    Lagu_2019[] dataLagu_2019 =
        new Lagu_2019[20];

    int jumlahData_2019 = 7;

    public void inputData_2019() {

        dataLagu_2019[0] =
            new Lagu_2019(
                "Monokrom",
                "Tulus",
                221);

        dataLagu_2019[1] =
            new Lagu_2019(
                "Iqro",
                "Hindia",
                245);

        dataLagu_2019[2] =
            new Lagu_2019(
                "Rabun Jauh",
                "Hindia",
                230);

        dataLagu_2019[3] =
            new Lagu_2019(
                "Perahu Kertas",
```

```

        "Maudy Ayunda",
        215);

    dataLagu_2019[4] =
        new Lagu_2019(
            "Runtuh",
            "Feby Putri",
            240);

    dataLagu_2019[5] =
        new Lagu_2019(
            "Indahnya Dirimu",
            "Naff",
            225);

    dataLagu_2019[6] =
        new Lagu_2019(
            "Lantas",
            "Juicy Luicy",
            255);
}

public void tampilData_2019() {

    for (int i_2019 = 0;
        i_2019 < jumlahData_2019;
        i_2019++) {

        System.out.println(
            (i_2019 + 1) + ". "
            + dataLagu_2019[i_2019].judul_2019
            + " - "
            + dataLagu_2019[i_2019].durasi_2019
            + " detik");

    }

}

public Lagu_2019[] copyArray_2019() {

    Lagu_2019[] temp_2019 =
        new Lagu_2019[jumlahData_2019];

    for (int i_2019 = 0;
        i_2019 < jumlahData_2019;
        i_2019++) {

        temp_2019[i_2019] =
            new Lagu_2019(
                dataLagu_2019[i_2019].judul_2019,
                dataLagu_2019[i_2019].penyanyi_2019,
                dataLagu_2019[i_2019].durasi_2019);

    }

    return temp_2019;

}

// SHELL SORT (JUDUL)

```

```

public void shellSort_2019(
    Lagu_2019[] arr_2019) {

    int n_2019 = arr_2019.length;

    for (int gap_2019 = n_2019 / 2;
        gap_2019 > 0;
        gap_2019 /= 2) {

        for (int i_2019 = gap_2019;
            i_2019 < n_2019;
            i_2019++) {

            Lagu_2019 temp_2019 =
                arr_2019[i_2019];

            int j_2019;

            for (j_2019 = i_2019;
                j_2019 >= gap_2019
                &&
                arr_2019[j_2019-gap_2019]
                    .judul_2019
                    .compareToIgnoreCase(
                        temp_2019.judul_2019)
                        > 0;
                j_2019 -= gap_2019) {

                arr_2019[j_2019] =
                    arr_2019[j_2019-gap_2019];
            }

            arr_2019[j_2019] =
                temp_2019;
        }
    }
}

// QUICK SORT (DURASI)

public int partition_2019(
    Lagu_2019[] arr_2019,
    int low_2019,
    int high_2019) {

    int pivot_2019 =
        arr_2019[high_2019].durasi_2019;

    int i_2019 =
        low_2019 - 1;

    for (int j_2019 = low_2019;
        j_2019 < high_2019;
        j_2019++) {

        if (arr_2019[j_2019]

```

```

        .durasi_2019
        <= pivot_2019) {

            i_2019++;

            Lagu_2019 temp_2019 =
                arr_2019[i_2019];

            arr_2019[i_2019] =
                arr_2019[j_2019];

            arr_2019[j_2019] =
                temp_2019;
        }
    }

    Lagu_2019 temp_2019 =
        arr_2019[i_2019 + 1];

    arr_2019[i_2019 + 1] =
        arr_2019[high_2019];

    arr_2019[high_2019] =
        temp_2019;

    return i_2019 + 1;
}

public void quickSort_2019(
    Lagu_2019[] arr_2019,
    int low_2019,
    int high_2019) {

    if (low_2019 < high_2019) {

        int pi_2019 =
            partition_2019(
                arr_2019,
                low_2019,
                high_2019);

        quickSort_2019(
            arr_2019,
            low_2019,
            pi_2019 - 1);

        quickSort_2019(
            arr_2019,
            pi_2019 + 1,
            high_2019);
    }
}

// MERGE SORT (JUDUL)

public void merge_2019(
    Lagu_2019[] arr_2019,

```



```

        int left_2019,
        int mid_2019,
        int right_2019) {

    int n1_2019 =
        mid_2019 - left_2019 + 1;

    int n2_2019 =
        right_2019 - mid_2019;

    Lagu_2019[] L_2019 =
        new Lagu_2019[n1_2019];

    Lagu_2019[] R_2019 =
        new Lagu_2019[n2_2019];

    for (int i_2019 = 0;
        i_2019 < n1_2019;
        i_2019++) {

        L_2019[i_2019] =
            arr_2019[left_2019+i_2019];
    }

    for (int j_2019 = 0;
        j_2019 < n2_2019;
        j_2019++) {

        R_2019[j_2019] =
            arr_2019[mid_2019+1+j_2019];
    }

    int i_2019 = 0;
    int j_2019 = 0;
    int k_2019 = left_2019;

    while (i_2019 < n1_2019
        && j_2019 < n2_2019) {

        if (L_2019[i_2019]
            .judul_2019
            .compareToIgnoreCase(
                R_2019[j_2019]
                .judul_2019)
            <= 0) {

            arr_2019[k_2019++] =
                L_2019[i_2019++];
        }

        else {

            arr_2019[k_2019++] =
                R_2019[j_2019++];
        }
    }
}

```

```

        while (i_2019 < n1_2019) {
            arr_2019[k_2019++] =
                L_2019[i_2019++];
        }

        while (j_2019 < n2_2019) {
            arr_2019[k_2019++] =
                R_2019[j_2019++];
        }
    }

    public void mergeSort_2019(
        Lagu_2019[] arr_2019,
        int left_2019,
        int right_2019) {

        if (left_2019 < right_2019) {

            int mid_2019 =
                (left_2019 + right_2019) / 2;

            mergeSort_2019(
                arr_2019,
                left_2019,
                mid_2019);

            mergeSort_2019(
                arr_2019,
                mid_2019 + 1,
                right_2019);

            merge_2019(
                arr_2019,
                left_2019,
                mid_2019,
                right_2019);
        }
    }

    public static void main(String[] args) {

        Scanner input_2019 =
            new Scanner(System.in);

        Sorting_2511532019 obj_2019 =
            new Sorting_2511532019();

        obj_2019.inputData_2019();

        int pilih_2019;

        do {

            System.out.println();
            System.out.println(
                "=== Sorting Playlist NIM: 2511532019 ===");
            System.out.print(

```

```

        "(1= Shell Sort, ");
System.out.print(
    "2= Quick Sort, ");
System.out.print(
    "3= Merge Sort, ");
System.out.println(
    "0= Keluar");

System.out.println(
    "Pilih Algoritma : ");

pilih_2019 =
    input_2019.nextInt();

Lagu_2019[] temp_2019 =
    obj_2019.copyArray_2019();

if (pilih_2019 == 1) {

    System.out.println(
        "\nData Sebelum Sorting:");
    obj_2019.tampilData_2019();

    obj_2019.shellSort_2019(
        temp_2019);

    System.out.println(
        "\nData Setelah Shell Sort:");

    for (int i_2019=0;
        i_2019<temp_2019.length;
        i_2019++) {

        System.out.println(
            (i_2019+1)+". "
            +temp_2019[i_2019]
            .judul_2019
            +" - "
            +temp_2019[i_2019]
            .durasi_2019
            +" detik");
    }
}

else if (pilih_2019 == 2) {

    System.out.println(
        "\nData Sebelum Sorting:");
    obj_2019.tampilData_2019();

    obj_2019.quickSort_2019(
        temp_2019,
        0,
        temp_2019.length-1);

    System.out.println(
        "\nData Setelah Quick Sort (Durasi Asc):");

```

```

        for (int i_2019=0;
            i_2019<temp_2019.length;
            i_2019++) {

            System.out.println(
                (i_2019+1)+". "
                +temp_2019[i_2019]
                .judul_2019
                +" - "
                +temp_2019[i_2019]
                .durasi_2019
                +" detik");

        }
    }

    else if (pilih_2019 == 3) {

        System.out.println(
            "\nData Sebelum Sorting:");
        obj_2019.tampilData_2019();

        obj_2019.mergeSort_2019(
            temp_2019,
            0,
            temp_2019.length-1);

        System.out.println(
            "\nData Setelah Merge Sort:");

        for (int i_2019=0;
            i_2019<temp_2019.length;
            i_2019++) {

            System.out.println(
                (i_2019+1)+". "
                +temp_2019[i_2019]
                .judul_2019
                +" - "
                +temp_2019[i_2019]
                .durasi_2019
                +" detik");

        }
    }

} while (pilih_2019 != 0);

System.out.println(
    "\nProgram selesai.");

input_2019.close();
}
}

```

- `import java.util.Scanner;`
Mengimpor class Scanner yang digunakan untuk menerima input dari pengguna melalui keyboard.
- `class Lagu_2019`
Membuat class Lagu yang digunakan untuk menyimpan data lagu.
- `String judul_2019;`
Menyimpan judul lagu.
- `String penyanyi_2019;`
Menyimpan nama penyanyi lagu.
- `int durasi_2019;`
Menyimpan durasi lagu dalam satuan detik.
- `public Lagu_2019(String judul_2019, String penyanyi_2019, int durasi_2019)`
Constructor yang digunakan untuk menginisialisasi data lagu.
- `this.judul_2019 = judul_2019;`
Mengisi atribut judul lagu.
- `this.penyanyi_2019 = penyanyi_2019;`
Mengisi atribut penyanyi lagu.
- `this.durasi_2019 = durasi_2019;`
Mengisi atribut durasi lagu.
- `public class Sorting_2511532019`
Membuat class utama program sorting playlist lagu.
- `Lagu_2019[] dataLagu_2019 = new Lagu_2019[20];`
Membuat array yang dapat menyimpan maksimal 20 data lagu.
- `int jumlahData_2019 = 7;`
Menentukan jumlah lagu yang digunakan sebanyak 7 data.
- `public void inputData_2019()`
Method untuk mengisi data lagu ke dalam array.
- `dataLagu_2019[0] = new Lagu_2019("Monokrom","Tulus",221);`
Menambahkan lagu Monokrom ke dalam array.
- `dataLagu_2019[1] = new Lagu_2019("Iqro","Hindia",245);`
Menambahkan lagu Iqro ke dalam array.

- `dataLagu_2019[2] = new Lagu_2019("Rabun Jauh","Hindia",230);`
Menambahkan lagu Rabun Jauh ke dalam array.
- `dataLagu_2019[3] = new Lagu_2019("Perahu Kertas","Maudy Ayunda",215);`
Menambahkan lagu Perahu Kertas ke dalam array.
- `dataLagu_2019[4] = new Lagu_2019("Runtuh","Feby Putri",240);`
Menambahkan lagu Runtuh ke dalam array.
- `dataLagu_2019[5] = new Lagu_2019("Indahnya Dirimu","Naff",225);`
Menambahkan lagu Indahnya Dirimu ke dalam array.
- `dataLagu_2019[6] = new Lagu_2019("Lantas","Juicy Luicy",255);`
Menambahkan lagu Lantas ke dalam array.
- `public void tampilData_2019()`
Method untuk menampilkan seluruh data lagu.
- `for (int i_2019 = 0; i_2019 < jumlahData_2019; i_2019++)`
Melakukan perulangan untuk menampilkan semua data lagu.
- `System.out.println(...)`
Menampilkan nomor urut, judul lagu, dan durasi lagu.
- `public Lagu_2019[] copyArray_2019()`
Method untuk membuat salinan array sebelum dilakukan sorting.
- `Lagu_2019[] temp_2019 = new Lagu_2019[jumlahData_2019];`
Membuat array sementara untuk menampung salinan data.
- `for (int i_2019 = 0; i_2019 < jumlahData_2019; i_2019++)`
Melakukan perulangan untuk menyalin seluruh data lagu.
- `temp_2019[i_2019] = new Lagu_2019(...)`
Menyalin data lagu ke array baru.
- `return temp_2019;`
Mengembalikan hasil salinan array.
- `public void shellSort_2019(Lagu_2019[] arr_2019)`
Method Shell Sort untuk mengurutkan lagu berdasarkan judul secara alfabet.
- `int n_2019 = arr_2019.length;`
Mengambil jumlah data dalam array.

- `for (int gap_2019 = n_2019 / 2; gap_2019 > 0; gap_2019 /= 2)`
Menentukan nilai gap yang digunakan pada Shell Sort.
- `Lagu_2019 temp_2019 = arr_2019[i_2019];`
Menyimpan sementara data yang akan dipindahkan.
- `compareToIgnoreCase()`
Membandingkan judul lagu tanpa membedakan huruf besar dan kecil.
- `arr_2019[j_2019] = arr_2019[j_2019-gap_2019];`
Menggeser data ke posisi berikutnya.
- `arr_2019[j_2019] = temp_2019;`
Menempatkan data pada posisi yang sesuai.
- `public int partition_2019(Lagu_2019[] arr_2019, int low_2019, int high_2019)`
Method pembantu Quick Sort yang menentukan posisi pivot.
- `int pivot_2019 = arr_2019[high_2019].durasi_2019;`
Menjadikan durasi lagu terakhir sebagai pivot.
- `int i_2019 = low_2019 - 1;`
Menentukan indeks awal elemen yang lebih kecil dari pivot.
- `for (int j_2019 = low_2019; j_2019 < high_2019; j_2019++)`
Melakukan perbandingan seluruh elemen terhadap pivot.
- `if (arr_2019[j_2019].durasi_2019 <= pivot_2019)`
Mengecek apakah durasi lagu lebih kecil atau sama dengan pivot.
- `Lagu_2019 temp_2019 = arr_2019[i_2019];`
Menyimpan sementara data saat proses pertukaran.
- `arr_2019[i_2019] = arr_2019[j_2019];`
Menukar posisi data.
- `arr_2019[j_2019] = temp_2019;`
Menyelesaikan proses pertukaran data.
- `arr_2019[i_2019 + 1] = arr_2019[high_2019];`
Menempatkan pivot pada posisi yang benar.
- `return i_2019 + 1;`
Mengembalikan indeks pivot.

- `public void quickSort_2019(Lagu_2019[] arr_2019, int low_2019, int high_2019)`
Method Quick Sort untuk mengurutkan lagu berdasarkan durasi.
- `if (low_2019 < high_2019)`
Memastikan array masih dapat dibagi.
- `int pi_2019 = partition_2019(...)`
Menentukan posisi pivot menggunakan method partition.
- `quickSort_2019(arr_2019, low_2019, pi_2019 - 1);`
Mengurutkan bagian kiri pivot.
- `quickSort_2019(arr_2019, pi_2019 + 1, high_2019);`
Mengurutkan bagian kanan pivot.
- `public void merge_2019(Lagu_2019[] arr_2019, int left_2019, int mid_2019, int right_2019)`
Method untuk menggabungkan dua sub-array yang telah terurut pada Merge Sort.
- `int n1_2019 = mid_2019 - left_2019 + 1;`
Menentukan ukuran array kiri.
- `int n2_2019 = right_2019 - mid_2019;`
Menentukan ukuran array kanan.
- `Lagu_2019[] L_2019 = new Lagu_2019[n1_2019];`
Membuat array sementara bagian kiri.
- `Lagu_2019[] R_2019 = new Lagu_2019[n2_2019];`
Membuat array sementara bagian kanan.
- `L_2019[i_2019] = arr_2019[left_2019+i_2019];`
Menyalin data ke array kiri.
- `R_2019[j_2019] = arr_2019[mid_2019+1+j_2019];`
Menyalin data ke array kanan.
- `while (i_2019 < n1_2019 && j_2019 < n2_2019)`
Membandingkan elemen array kiri dan kanan.
- `compareToIgnoreCase()`
Membandingkan judul lagu secara alfabet.

- `arr_2019[k_2019++] = L_2019[i_2019++];`
Menyalin elemen dari array kiri ke array utama.
- `arr_2019[k_2019++] = R_2019[j_2019++];`
Menyalin elemen dari array kanan ke array utama.
- `while (i_2019 < n1_2019)`
Menyalin sisa elemen pada array kiri.
- `while (j_2019 < n2_2019)`
Menyalin sisa elemen pada array kanan.
- `public void mergeSort_2019(Lagu_2019[] arr_2019, int left_2019, int right_2019)`
Method utama Merge Sort untuk mengurutkan data berdasarkan judul.
- `int mid_2019 = (left_2019 + right_2019) / 2;`
Menentukan titik tengah array.
- `mergeSort_2019(arr_2019, left_2019, mid_2019);`
Mengurutkan bagian kiri array.
- `mergeSort_2019(arr_2019, mid_2019 + 1, right_2019);`
Mengurutkan bagian kanan array.
- `merge_2019(arr_2019, left_2019, mid_2019, right_2019);`
Menggabungkan kedua bagian yang sudah terurut.
- `public static void main(String[] args)`
Method utama untuk menjalankan program.
- `Scanner input_2019 = new Scanner(System.in);`
Membuat objek Scanner untuk menerima input pengguna.
- `Sorting_2511532019 obj_2019 = new Sorting_2511532019();`
Membuat objek program sorting.
- `obj_2019.inputData_2019();`
Mengisi data lagu ke dalam array.
- `do { ... } while (pilih_2019 != 0);`
Menampilkan menu secara berulang sampai pengguna memilih keluar.
- `pilih_2019 = input_2019.nextInt();`
Membaca pilihan algoritma dari pengguna.

- `Lagu_2019[] temp_2019 = obj_2019.copyArray_2019();`
Membuat salinan data sebelum sorting.
- `if (pilih_2019 == 1)`
Menjalankan proses Shell Sort.
- `obj_2019.shellSort_2019(temp_2019);`
Memanggil algoritma Shell Sort.
- `else if (pilih_2019 == 2)`
Menjalankan proses Quick Sort.
- `obj_2019.quickSort_2019(temp_2019, 0, temp_2019.length-1);`
Memanggil algoritma Quick Sort.
- `else if (pilih_2019 == 3)`
Menjalankan proses Merge Sort.
- `obj_2019.mergeSort_2019(temp_2019, 0, temp_2019.length-1);`
Memanggil algoritma Merge Sort.
- `System.out.println("Data Sebelum Sorting");`
Menampilkan data playlist sebelum diurutkan.
- `System.out.println("Data Setelah Sorting");`
Menampilkan data playlist setelah diurutkan.
- `System.out.println("\nProgram selesai.");`
Menampilkan pesan bahwa program telah selesai dijalankan.
- `input_2019.close();`
Menutup Scanner dan mengakhiri penggunaan input keyboard.

Output

```
=== Sorting Playlist NIM: 2511532019 ===  
(1= Shell Sort, 2= Quick Sort, 3= Merge Sort, 0= Keluar)  
Pilih Algoritma :
```

```
1
```

```
Data Sebelum Sorting:
```

1. Monokrom - 221 detik
2. Iqro - 245 detik
3. Rabun Jauh - 230 detik
4. Perahu Kertas - 215 detik
5. Runtuh - 240 detik
6. Indahnya Dirimu - 225 detik
7. Lantas - 255 detik

```
Data Setelah Shell Sort:
```

1. Indahnya Dirimu - 225 detik
2. Iqro - 245 detik
3. Lantas - 255 detik
4. Monokrom - 221 detik
5. Perahu Kertas - 215 detik
6. Rabun Jauh - 230 detik
7. Runtuh - 240 detik

```
=== Sorting Playlist NIM: 2511532019 ===  
(1= Shell Sort, 2= Quick Sort, 3= Merge Sort, 0= Keluar)  
Pilih Algoritma :
```

```
2
```

```
Data Sebelum Sorting:
```

1. Monokrom - 221 detik
2. Iqro - 245 detik
3. Rabun Jauh - 230 detik
4. Perahu Kertas - 215 detik
5. Runtuh - 240 detik
6. Indahnya Dirimu - 225 detik
7. Lantas - 255 detik

```
Data Setelah Quick Sort (Durasi Asc):
```

1. Perahu Kertas - 215 detik
2. Monokrom - 221 detik
3. Indahnya Dirimu - 225 detik
4. Rabun Jauh - 230 detik
5. Runtuh - 240 detik
6. Iqro - 245 detik
7. Lantas - 255 detik

```
=== Sorting Playlist NIM: 2511532019 ===  
(1= Shell Sort, 2= Quick Sort, 3= Merge Sort, 0= Keluar)  
Pilih Algoritma :  
3
```

```
Data Sebelum Sorting:  
1. Monokrom - 221 detik  
2. Iqro - 245 detik  
3. Rabun Jauh - 230 detik  
4. Perahu Kertas - 215 detik  
5. Runtuh - 240 detik  
6. Indahnya Dirimu - 225 detik  
7. Lantas - 255 detik
```

```
Data Setelah Merge Sort:  
1. Indahnya Dirimu - 225 detik  
2. Iqro - 245 detik  
3. Lantas - 255 detik  
4. Monokrom - 221 detik  
5. Perahu Kertas - 215 detik  
6. Rabun Jauh - 230 detik  
7. Runtuh - 240 detik
```

```
=== Sorting Playlist NIM: 2511532019 ===  
(1= Shell Sort, 2= Quick Sort, 3= Merge Sort, 0= Keluar)  
Pilih Algoritma :  
0
```

```
Program selesai.
```